CMPE322

2025 FALL PROJECT-2

1- INTRODUCTION

In this project you will implement a hashing algorithm with multiple threads. Two different hashing approach is given to you as a quest. By their nature they may speed up, or fall into the deadlock. The project aims you to measure their performance and solve the issues.

2- HASH ALGORITHMS

In this project you will implement two different hashing approaches. The given functions must be implemented separately. The notation of these two functions are given below:

n	:	Size of the hash array.
m	:	Number of the entries to be placed to the hash array.
t	:	Number of threads.
h_x	:	Hash function.
c	:	Number of collusions
i	:	Currently selected entry.
k	:	Second hash function coefficient, a fixed value, n mod k ≡ 0.

$h\_1(i) = ((i + c) \bmod n)$.

$h\_2(i) \in \bigcup [((i + c) \bmod k) + zk]$, $z \in \{0, 1, 2, \ldots (n/k)-1\}$.

The first hash function checks the hash array's (i mod n)'th index. If that slot is empty, then the value i is replaced to that slot. Otherwise, it is called a collusion and the next slot is checked, which is ((i + 1) mod n). In this case "c" is equal to 1. If that slot is available, then put the given value there, if not, then c becomes 2 and check ((i + 2) mod n). This procedure goes on like that.

The second hash function is a bit complicated to explain, but easy. The hash array is split to (n / k) windows each having the length of k. The function puts if any of the window's ((i + c) mod k) slot is available, then puts it. No order is declared. If no window with empty slot is found at the first attempt, then the function will increase c (the collusion number) by one and retries the all procedure. If it cannot find again, increases c by 1 again and continues until a valid slot is found.

3- SEQUENTIAL APPROACH

In this section pseudocode of the hashing algorithms mentioned in the sequential manner.

For h_1:

```
S1    i ← First entry
S2    c ← 0
S3    While not processed entries are existing:
S4        Check hash_array[(i + c) mod n].
S5        If empty:
S6            Put i there.
S7        Else:
S8            c ← c + 1
S9            Go to state S4.
S10       Mark entry i as processed.
S11       i ← next entry.
S12       c ← 0
S13   Finish the procedure.
```

For h_2:

```
S1    i ← First entry
S2    c ← 0
S3    While not processed entries are existing:
S4        For all values of z < (n/k):
S5            Check hash_array[((i + c) mod k) + zk].
S6            If empty slot is found:
S7                Put i there.
S8            Else:
S9                c ← c + 1
S10               Go to state S4.
S11       Mark entry i as processed.
S12       i ← next entry.
S13       c ← 0
S14   Finish the procedure.
```

4- PARALLEL APPROACH

In parallel programming approach the values are put in the table similarly. For the parallel case, there will be differences among the h_1 and h_2 functions in terms of acquiring / releasing locks and speedups.

To do that allocate n locks.

For h_1:

For each thread

```
S1    i ← First entry of thread's possessed entry group
S2    c ← 0
S3    While not processed entries in thread's possessed entry group are existing:
S4          Try acquiring Locks[(i + c) mod n].
S5          If succeded:
S6                Check hash_array[(i + c) mod n].
S7                If empty:
S8                      Put i there.
S9                      Release Locks[(i + c) mod n].
S10               Else:
S11                     Release Locks[(i + c) mod n].
S12                     c ← c + 1
S13                     Go to state S4
S14               Release Locks[(i + c) mod n].
S15         Else:
S16               Go to state S4.
S17         Mark entry i as processed.
S18         i ← next entry.
S19         c ← 0
S20   Finish the procedure.
```

For h_2:

```
        For each thread

S1      i ← First entry of thread's possessed entry group
S2      c ← 0
S3      While not processed entries in thread's possessed entry group are existing:
S4              rand_val ← get_random_val()  // rand_val < k.
S5              Set up a counter cnt.
S6              cnt ← 0
S7              Do (n / k) times:
S8                      Try acquiring Locks[ ((i + c) mod k) + ((rand_val + cnt) mod (n/k)) * k ]
S9                      If failed:
S10                             Release all acquired locks.
S11                             Go to state S4
S12                     cnt ← cnt + 1
S13             cnt ← 0
S14             While cnt < (n / k):
S15                     Check hash_array[ ((i + c) mod k) + ((rand_val + cnt) mod (n/k)) * k ].
S16                     If empty:
S17                             Put i there.
S18                             Release all acquired locks.
S19                     cnt ← cnt + 1
S20             If no available slot is found in the previous loop:
S21                     Release all acquired locks.
S22                     c ← c + 1
S23                     Go to state S4.
S24             Mark entry i as processed.
S25             i ← next entry.
S26             c ← 0
S27     Finish the procedure.
```

Inspect those two algorithms which are required to be implemented using POSIX Thread library (pthread). First algorithm focuses on speedup while the second one focuses on keeping you in deadlock state. Therefore, you have to deal with deadlock situations and handle. Handling procedure will be given to you.

5- SPEEDUP AND DEADLOCKS

Parallel programming has immense advantage which speeds up the programs, but everything comes with a price: extra checks and synchronization. Otherwise deadlocks or race conditions happen.

h_1 algorithm focuses on pure speedup with a basic hash mechanism, it checks, then tries to lock. If everything goes well, it will speed up immensely.

h_2 algorithm has a wrong design by nature, firstly it locks, then checks whether the possible slots are empty, and creates a suitable environment for deadlocks.

For h_1, measure the speedup. Use “sequential_time / parallel_time”. Put the result in ‘h_1_speedup’ value.

For h_2, prevent possible deadlocks. While taking the locks, if a collusion happens and a thread detects that, release all the locks so far (State S10). To prevent deadlocks DO NOT implement a timing mechanism or other sophisticated techniques, just release the locks as described. Your implementation MUST NOT fall into the deadlock state. You have to deal with these in a reasonable time, no sleeps, no waits, no random time-ups.

6- DATA STRUCTURES

In this project you will have a “hash_parallelization.h” header file, which declares the functions you will implement and creates variables / pointers to use. They will be described below:

- struct entry_struct **hash_array
  - Actual hash array that holds the addresses of the given entries. Its size is n. After allocation all the members of this array should be set to 0 (NULL).
- struct entry_struct
  - Has two fields: value and timestamp.
  - int value: A unique integer value that will be used in hash function. Structs will be placed to the array according to the value.
  - clock_t timestamp: A timestamp which must be assigned at the time when that struct is successfully placed on the array. There is no need to be in seconds or other formats, just place the clock function’s value. Check the clock() function in C.
- struct entry_struct *entry_list
  - A pointer that must hold a dynamically allocated array of entry_struct’s. Its size is m. Check malloc function.
- pthread_mutex_t *lock_list
  - A pointer that must hold a dynamically allocated array of mutex locks (pthread_mutex_t). Its size is n. Check malloc function.
- double h_1_speedup
  - Speed-up value for h_1. Measure the time and put the speedup value.
- int n
  - Size of the hash array.
- int m
  - Number of values to be placed on hash array.
- int t
  - Number of threads. Number m will be evenly divisible by t.
- int k
  - The special value of h_2. Determines the length of the window in hash array that a value should be put on it. Number n will be evenly divisible by k.
- int random_seed
  - Random seed value used in srand() function.

7- TASK

Use POSIX Threads library (pthread) for multithread coding. Any other library WILL NOT BE ACCEPTED.

Implement the array_allocation() function. It must be used only to allocate arrays and put their pointer values on entry_list and lock_list. Check malloc() function. It is there to split the preparation – non-parallel – phase of the computation.

```
array_allocation() {
        allocate hash_array array. (pointer struct entry_struct array)
        allocate entry_list array. (struct entry_struct array)
        allocate lock_list array. (mutex lock array)
        Set up locks.
        Return.
}

array_deallocation() {
        Destroy locks.
        Release hash_array array.
        release entry_list array.
        release lock_list array.
        Return.
}
```

You will implement parallel_h_1, parallel_h_2, sequential_h_1 functions. Function declarations will be found in header file. Contents of the algorithms are described in detail previously.

For parallel_h_1 and parallel_h_2:

Create t threads, each thread should deal with (m / t) entries. First thread should deal with 0 to (m / t) - 1 indices of the entry_list, second thread should deal with (m / t) to 2 * (m / t) – 1 indices, and so on. The last thread should be dealing with the last (m / t) indices.

For the speedup comparison implement the speedup_comparison_h_1() function. Call parallel h1 and sequential h1 with measuring their execution times, then use “sequential_time / parallel_time” to find actual speedup and put the value in h_1_speedup variable.

For any of these h_x family of functions, after putting a value in the hash array update its timestamp with proper time value got from clock_gettime() function. DO NOT forget to call clock_gettime() and measure time. This code snippet below shows how to use clock properly:

```
        struct timespec start, end;
        clock_gettime(CLOCK_MONOTONIC, &start);
        // WORK //
        clock_gettime(CLOCK_MONOTONIC, &end);
        int64_t time_spent = // Use the timespec values to calculate time in nanoseconds
                                // precision.
```

Due to the randomness nature of the multithread programs there will be more than one correct solution. So the timestamps will provide us the ability of mimicking the procedure with sequential approach. WITHOUT TIMESTAMPS YOU WILL GET NO POINTS.

Your code must be run by calling these declared functions in the manner described below:

```
int main(void) {
        init();
        array_allocation();
        // Setting values, depends on you to select your testing values. //
        h_x family of functions;
        check_entry_list(); // You will not implement this.
        array_deallocation();
        return 0;
}
```

After implementing your functions, create a shared object file named hash_parallelization.so. DO NOT EDIT THE HEADER FILE, if you need extra functions or variables create those in your hash_parallelization.c file. Your program MUST BE fully functional by calling the functions described in the code snippet above, NO MORE NO LESS.

8- SUBMISSION AND REPORT

Submit a zip with name <your_student_id>.zip. This zip will contain hash_parallelization.so, hash_parallelization.c, and report.pdf. Just compress these files, not the folder containing it. After decompressing these files must be visible immediately, not in a folder or any other stuff.

Report MUST BE at most 2 pages long containing the link to the manual pages about built-in functions you used. The typical research topics are written in the header file as comments, and mentioned in this project pdf.

Put your GPT engine queries and their answers as a link to the conversation if any is used. Use English in a formal manner while chatting with GPT engines.

Compare two h_x functions in terms of speed and effectiveness. One paragraph with 3 sentences is enough. Use 12pt Times New Roman in your report.

For testing purposes, it is STRONGLY ADVICED (almost compulsory) to implement all sequential and parallel algorithms. Sequential algorithms can be used to test your parallel code using timestamp values.

Tests are automated; therefore, be careful about the file format and content you submit. A change in header file may cause you to take 0.

Good luck, this is an easy project. You have to just learn about how to program multithreaded programs. Algorithms are already given. Test cases may be provided.